

# Báo cáo bài tập: Cờ Caro với Minimax và Alpha-Beta Pruning

Thành viên thực hiện:

Vũ Ngọc Sơn (24022446)

Phạm Quang Tiến (24022464)

Phạm Thành Trung (24022476)

## 1 Mô tả bài toán

Cờ Caro là một trò chơi đối kháng hai người trên bàn cờ vuông. Hai người chơi luân phiên đặt quân của mình vào các ô trống. Bên nào tạo được một dãy liên tiếp đủ dài (theo hàng, cột hoặc đường chéo) trước thì thắng. Trong bài tập này:

- Bàn cờ kích thước  $9 \times 9$  (có thể mở rộng).
- Điều kiện thắng: **4 quân liên tiếp**.
- Người chơi (X) là MIN, máy (O) là MAX.
- Mục tiêu: máy chọn nước đi tốt nhất bằng **Minimax** (Level 1) và **Alpha-Beta Pruning** (Level 2), sau đó **so sánh thực nghiệm** (Level 3).

## 2 Luật chơi và điều kiện thắng

- Hai bên đánh luân phiên, mỗi lượt đặt đúng 1 quân vào ô trống.
- Không được đánh vào ô đã có quân.
- Ván kết thúc khi: (a) một bên có 4 quân liên tiếp, hoặc (b) bàn cờ đầy mà chưa ai thắng (hòa).

## 3 Biểu diễn trạng thái bàn cờ

Bàn cờ là ma trận `size × size` với 3 giá trị:

| Giá trị | Ý nghĩa     | Ký hiệu hiển thị |
|---------|-------------|------------------|
| 0       | Ô trống     | .                |
| 1       | HUMAN (MIN) | X                |
| 2       | AI (MAX)    | O                |

Lớp `CaroGame` đóng gói ma trận và các thao tác `make_move`, `undo_move`, `check_win`, `is_terminal`, `get_valid_moves`.

## 4 Sinh nước đi hợp lệ

Một bàn 9×9 có 81 ô, nếu xét toàn bộ ô trống thì branching factor ban đầu là 80 — quá lớn cho Minimax ở độ sâu 3+. Vì các nước đi cách xa mọi quân hiện có gần như không ảnh hưởng đến kết cục, chương trình chỉ sinh các ô trống **nằm trong bán kính 1 cạnh một quân đã đặt**:

```
1 def get_valid_moves(self):
2     if self.is_empty():
3         return [(self.size // 2, self.size // 2)]
4     moves = set()
5     for r in range(self.size):
6         for c in range(self.size):
7             if self.board[r][c] == EMPTY:
8                 continue
9             for dr in range(-1, 2):
10                for dc in range(-1, 2):
11                    nr, nc = r + dr, c + dc
12                    if self.in_bounds(nr, nc) and self.board[nr][nc] ==
13                        EMPTY:
14                        moves.add((nr, nc))
15     return sorted(moves)
```

Cách lọc này áp dụng cho cả Minimax và Alpha-Beta nên việc so sánh hai thuật toán vẫn công bằng.

## 5 Kiểm tra trạng thái kết thúc

Hàm check\_win(player) duyệt từng ô có quân của player, đi 4 hướng (ngang, dọc, 2 đường chéo) và đếm số quân liên tiếp. Nếu đếm được WIN\_LENGTH = 4 thì trả về True.

```
1 def is_terminal(self):
2     if self.check_win(AI): return True, WIN_SCORE # +100000
3     if self.check_win(HUMAN): return True, -WIN_SCORE # -100000
4     if self.is_full(): return True, 0
5     return False, 0
```

## 6 Thuật toán Minimax (Level 1)

```
1 def minimax(game, depth, is_max_turn, stats):
2     stats.states += 1
3     terminal, value = game.is_terminal()
4     if terminal:
5         return value, None
6     if depth == 0:
7         return evaluate(game), None
8
9     moves = game.get_valid_moves()
10    best_move = moves[0]
11
12    if is_max_turn:
13        best_val = float('-inf')
14        for r, c in moves:
15            game.make_move(r, c, AI)
16            val, _ = minimax(game, depth-1, False, stats)
17            game.undo_move(r, c)
```

```

18         if val > best_val:
19             best_val, best_move = val, (r, c)
20         return best_val, best_move
21     else:
22         best_val = float('inf')
23         for r, c in moves:
24             game.make_move(r, c, HUMAN)
25             val, _ = minimax(game, depth-1, True, stats)
26             game.undo_move(r, c)
27             if val < best_val:
28                 best_val, best_move = val, (r, c)
29         return best_val, best_move

```

Đặc điểm:

- Trạng thái kết thúc → trả về giá trị tuyệt đối ( $\pm \text{WIN\_SCORE} / 0$ ).
- Đạt độ sâu giới hạn → dùng evaluate.
- Lượt MAX chọn max, lượt MIN chọn min.
- Trả về cặp (giá trị, nước đi tốt nhất).

## 7 Thuật toán Alpha-Beta Pruning (Level 2)

```

1 def alphabeta(game, depth, alpha, beta, is_max_turn, stats):
2     stats.states += 1
3     terminal, value = game.is_terminal()
4     if terminal: return value, None
5     if depth == 0: return evaluate(game), None
6
7     moves = game.get_valid_moves()
8     best_move = moves[0]
9
10    if is_max_turn:
11        best_val = float('-inf')
12        for r, c in moves:
13            game.make_move(r, c, AI)
14            val, _ = alphabeta(game, depth-1, alpha, beta, False, stats)
15            game.undo_move(r, c)
16            if val > best_val:
17                best_val, best_move = val, (r, c)
18            alpha = max(alpha, best_val)
19            if beta <= alpha: # Beta cut-off
20                break
21        return best_val, best_move
22    else:
23        best_val = float('inf')
24        for r, c in moves:
25            game.make_move(r, c, HUMAN)
26            val, _ = alphabeta(game, depth-1, alpha, beta, True, stats)
27            game.undo_move(r, c)
28            if val < best_val:
29                best_val, best_move = val, (r, c)
30            beta = min(beta, best_val)
31            if beta <= alpha: # Alpha cut-off
32                break
33        return best_val, best_move

```

Ý nghĩa các biến:

- **alpha**: giá trị tốt nhất mà MAX có thể đảm bảo trên đường đi hiện tại.
- **beta**: giá trị tốt nhất mà MIN có thể đảm bảo trên đường đi hiện tại.
- Khi  $\beta \leq \alpha \rightarrow$  nhánh còn lại không thể ảnh hưởng đến kết quả, cắt.

Alpha-Beta dùng **cùng evaluate**, **cùng get\_valid\_moves** và **cùng** giới hạn **depth** với Minimax  $\rightarrow$  kết quả luôn trả về cùng giá trị tốt nhất, chỉ khác ở số trạng thái xét.

## 8 Hàm đánh giá trạng thái

```
1 def evaluate(game):  
2     return _score_for(game, AI) - _score_for(game, HUMAN)
```

`_score_for(player)` quét mọi cửa số độ dài 4 trên cả 4 hướng. Với mỗi cửa số **chỉ chứa quân của player hoặc ô trống** (cửa số vẫn còn cơ hội hoàn thành 4 quân), điểm được tính theo số quân của player:

| Số quân của player trong cửa sổ 4-ô | Điểm   |
|-------------------------------------|--------|
| 4 (đã thắng)                        | 100000 |
| 3                                   | 1000   |
| 2                                   | 100    |
| 1                                   | 10     |

Cửa sổ chứa quân của cả hai bên không được tính cho bên nào (không ai có thể hoàn thành 4 quân trong cửa sổ ấy nữa).

Điểm cuối cùng = **điểm AI** – **điểm HUMAN**, nên dương lợi cho MAX và âm lợi cho MIN — đúng quy ước Minimax.

## 9 Thiết kế các trạng thái thử nghiệm

| Mã | Tình huống                         | Mục đích kiểm thử       |
|----|------------------------------------|-------------------------|
| S1 | Đầu ván, mới đi 2-3 nước           | Bàn rộng, branching nhỏ |
| S2 | Giữa ván, AI có cơ hội 3-mở        | AI tấn công             |
| S3 | AI có 3 quân liên tiếp hở 2 đầu    | AI thắng ngay           |
| S4 | HUMAN có 3 quân liên tiếp hở 2 đầu | AI buộc phải chặn       |
| S5 | Cả hai bên đều có thế 2-mở         | Lựa chọn ưu tiên        |
| S6 | Quân rải rác toàn bàn              | Branching lớn nhất      |

(Bố cục chính xác xem trong file `experiment.py`.)

## 10 Bảng kết quả thực nghiệm

Chạy trên cùng máy, cùng `evaluate`, cùng `get_valid_moves`, mỗi cặp Minimax / Alpha-Beta dùng **cùng độ sâu**. `#st` = số trạng thái đã xét; `ms` = thời gian (mili giây); `cùng?` = hai thuật toán có chọn cùng nước đi không; `Cắt %` =  $1 - \text{states\_AB} / \text{states\_MM}$ ; `Speed-up` =  $\text{time\_MM} / \text{time\_AB}$ .

| Trạng thái           | D | MM move | MM #st | MM ms  | AB move | AB #st | AB ms | Cùng? | Cắt %         | Speed-up      |
|----------------------|---|---------|--------|--------|---------|--------|-------|-------|---------------|---------------|
| S1 Đầu ván           | 1 | (3,3)   | 13     | 7.5    | (3,3)   | 13     | 7.0   | ✓     | 0.0 %         | 1.08×         |
| S1 Đầu ván           | 2 | (3,3)   | 187    | 102.8  | (3,3)   | 40     | 16.2  | ✓     | 78.6 %        | 6.36×         |
| S1 Đầu ván           | 3 | (3,3)   | 3 121  | 1 684  | (3,3)   | 281    | 131   | ✓     | <b>91.0 %</b> | <b>12.85×</b> |
| S2 Giữa ván          | 1 | (2,4)   | 20     | 10.1   | (2,4)   | 20     | 9.9   | ✓     | 0.0 %         | 1.02×         |
| S2 Giữa ván          | 2 | (2,4)   | 380    | 191    | (2,4)   | 80     | 32.9  | ✓     | 78.9 %        | 5.82×         |
| S2 Giữa ván          | 3 | (2,4)   | 7 982  | 4 009  | (2,4)   | 777    | 371   | ✓     | 90.3 %        | 10.80×        |
| S3 AI thắng ngay     | 1 | (4,2)   | 17     | 8.5    | (4,2)   | 17     | 8.2   | ✓     | 0.0 %         | 1.03×         |
| S3 AI thắng ngay     | 2 | (4,2)   | 271    | 148    | (4,2)   | 120    | 61.7  | ✓     | 55.7 %        | 2.41×         |
| S3 AI thắng ngay     | 3 | (2,2)   | 5 415  | 2 718  | (2,2)   | 605    | 301   | ✓     | 88.8 %        | 9.02×         |
| S4 Phải chặn         | 1 | (4,6)   | 20     | 11.6   | (4,6)   | 20     | 11.0  | ✓     | 0.0 %         | 1.05×         |
| S4 Phải chặn         | 2 | (1,3)   | 418    | 211    | (1,3)   | 277    | 141   | ✓     | 33.7 %        | 1.49×         |
| S4 Phải chặn         | 3 | (1,3)   | 8 690  | 4 820  | (1,3)   | 2 181  | 1 128 | ✓     | 74.9 %        | 4.27×         |
| S5 Hai bên đều mạnh  | 1 | (5,4)   | 23     | 13.4   | (5,4)   | 23     | 13.1  | ✓     | 0.0 %         | 1.03×         |
| S5 Hai bên đều mạnh  | 2 | (5,4)   | 547    | 312    | (5,4)   | 258    | 140   | ✓     | 52.8 %        | 2.23×         |
| S5 Hai bên đều mạnh  | 3 | (2,1)   | 13 910 | 7 710  | (2,1)   | 1 929  | 1 025 | ✓     | 86.1 %        | 7.52×         |
| S6 Nhiều nước hợp lệ | 1 | (4,4)   | 44     | 26.0   | (4,4)   | 44     | 26.9  | ✓     | 0.0 %         | 0.97×         |
| S6 Nhiều nước hợp lệ | 2 | (4,4)   | 1 911  | 1 089  | (4,4)   | 1 001  | 560   | ✓     | 47.6 %        | 1.94×         |
| S6 Nhiều nước hợp lệ | 3 | (4,4)   | 83 749 | 48 001 | (4,4)   | 16 101 | 9 097 | ✓     | 80.8 %        | 5.28×         |

## 11 Nhận xét về số trạng thái và thời gian

- **Alpha-Beta luôn chọn cùng nước đi như Minimax** ở mọi trạng thái và mọi độ sâu đã thử (cột “Cùng?” toàn ✓). Điều này khớp với lý thuyết: cắt nhánh chỉ loại bỏ những nhánh chắc chắn không tốt hơn, không thay đổi giá trị tối ưu của gốc.
- **Tỷ lệ cắt phụ thuộc mạnh vào độ sâu:**
  - Độ sâu 1: hai thuật toán xét **gần như cùng số trạng thái** (cắt  $\approx 0\%$ ) vì Alpha-Beta chỉ thật sự có lợi khi có nhiều tầng quay lui.
  - Độ sâu 2: cắt 30 % – 80 %.
  - Độ sâu 3: cắt **75 % – 91 %**, speed-up đạt **4×–13×**.
- **Tỷ lệ cắt phụ thuộc vào tính chất trạng thái:**
  - Các trạng thái rõ ràng (S1, S2) có cắt cao nhất ( $\sim 90\%$ ) vì giá trị tối ưu sớm ổn định, alpha/beta thu hẹp nhanh.
  - Trạng thái phức tạp / cân bằng (S4 Phải chặn, S5 Hai bên đều mạnh) cắt thấp hơn vì các nhánh khó loại trừ sớm.
- **Thời gian tăng nhanh theo độ sâu:** Minimax ở S6 chạy 48 giây ở độ sâu 3 (so với 1 giây ở độ sâu 2  $\rightarrow$  tỷ lệ  $\sim 50\times$ , gần với branching factor trung bình). Alpha-Beta giảm xuống còn  $\sim 9$  giây.

## 12 Nhận xét về ảnh hưởng của độ sâu

- **Độ sâu 1:** AI chỉ thấy “một nước trước mắt”, thường bỏ qua các nước phòng thủ cần thiết. Ví dụ S4 ở độ sâu 1, AI chọn (4,6) (đánh cạnh quân X) thay vì nhận ra mình thua không thể cứu — giá trị heuristic +60 đánh lừa AI.
- **Độ sâu 2:** AI bắt đầu nhìn ra tình huống bị buộc thắng/thua. Ở S4, AI nhận ra HUMAN có 3-mở hai đầu — không cách nào chặn cả hai  $\rightarrow$  mọi nước đi đều dẫn đến  $-100000$ , AI chọn nước đầu tiên trong danh sách.

- **Độ sâu 3:** AI nhìn xa hơn, ở S1 đã phát hiện được chuỗi nước dẫn tới thắng (+100000). Tuy nhiên chi phí tăng mạnh — đó là lý do **Alpha-Beta trở nên thực sự cần thiết** ở độ sâu  $\geq 3$ .
- **Hiện tượng “thắng/thua chắc chắn”:** khi nhiều nước đi cùng dẫn đến cùng giá trị  $\pm \text{WIN\_SCORE}$ , thuật toán chọn nước đầu tiên trong danh sách (do **sorted**). Đó là lý do S3 độ sâu 3, AI chọn (2,2) thay vì (4,2)/(4,6) — cả ba đều có giá trị 100000 nhìn đủ sâu vì đi đâu AI cũng thắng.

## 13 Ưu điểm và hạn chế

### Ưu điểm

- Cài đặt rõ ràng, tách biệt logic bàn cờ và logic tìm kiếm.
- Cùng `evaluate` và `get_valid_moves` được Minimax và Alpha-Beta dùng chung  $\rightarrow$  so sánh công bằng.
- Hàm `get_valid_moves` lọc theo vùng lân cận quân đã đặt giúp giảm branching ban đầu từ 80 xuống còn 10–30.
- Alpha-Beta giảm 50 %–90 % số trạng thái ở độ sâu  $\geq 2$ .

### Hạn chế

- Heuristic đơn giản chỉ đếm số quân trong cửa sổ, **không phân biệt** mẫu “mở hai đầu” và “bị chặn một đầu” — đây là điểm yếu rõ nhất.
- Không có **sắp xếp nước đi** (move ordering) trước khi vào Alpha-Beta; thứ tự duyệt thuận theo (r, c), nếu sắp theo điểm heuristic trước thì tỷ lệ cắt sẽ còn cao hơn nữa.
- Khi gặp tình huống “thua chắc”, AI chọn nước đầu tiên thay vì nước cố gắng kéo dài ván — gây hành vi trông kỳ lạ.
- Độ sâu vẫn còn nông; với điều kiện thắng 4 quân, sâu hơn 3 sẽ rất chậm nếu không có move ordering / transposition table.

## 14 Hướng cải tiến

1. **Heuristic mạnh hơn:** phân biệt mẫu mở/đóng, ví dụ `_000_` (mở hai đầu)  $>$  `X000_` (mở một đầu).
2. **Move ordering:** sắp xếp nước đi theo điểm `evaluate` giảm dần trước khi gọi đệ quy  $\rightarrow$  Alpha-Beta cắt sớm hơn.
3. **Iterative deepening:** tăng dần độ sâu, dùng kết quả độ sâu trước để xếp thứ tự cho độ sâu sau.
4. **Transposition table:** ghi nhớ các trạng thái đã đánh giá để tránh tính lại.
5. **Threat-space search:** tìm chuỗi đe dọa bắt buộc, đặc biệt hiệu quả với Caro.

## 15 Hướng dẫn chạy

```
1 # Chơi tương tác với máy  
2 python3 play.py  
3  
4 # Chạy thực nghiệm Level 3 và in bảng kết quả  
5 python3 experiment.py
```

`experiment.py` cũng lưu toàn bộ số liệu chi tiết vào `results.json`.

## 16 Tài liệu tham khảo

- Stuart Russell, Peter Norvig — *Artificial Intelligence: A Modern Approach*, chương “Adversarial Search” (Minimax, Alpha-Beta).
- Bài giảng môn Trí tuệ Nhân tạo (slides về Game tree, Minimax, Alpha-Beta).
- Tham khảo cách tổ chức module (game state / search / evaluate tách rời) từ các repo Gomoku/Caro Python mã nguồn mở.